

Systems programming languages such as C and C++ offer the programmer granular control over hardware and efficient use of resources, making them appropriate for performance-critical applications. However, these languages are fundamentally memory-unsafe and can lead to security vulnerabilities that go unnoticed in production code. For example, the Heartbleed bug [1] was the result of an out-of-bounds read, which led to sensitive data being leaked.

There are several mechanisms that modern languages implement to address problems such as out-of-bounds access. Languages such as Python and Java use runtime checks and garbage collection to ensure memory safety guarantees, but these methods incur overhead that can degrade performance. The Rust programming language enforces compile-time safety using language features such as ownership and borrow-checking to prevent undefined behaviour (e.g. use-after-free and double free) that is prevalent in C and C++.

While Rust does provide a strong solution for preserving safety guarantees while maintaining performance comparable to C and C++, its correctness relies on the assumption that the compiler implementation is correct and preserves the semantics of the high-level (source) language. However, there may be situations in which a compiler for some given source language is not trusted. Projects such as CompCert [2] and CakeML [3] have developed verified compilers that ensure compiled programs behave exactly as the source program was defined.

This dissertation project aims to move the locus of trust from the compiler to a simple, independent verifier. Formal compiler verification is complex and requires a lot of work whereas a verifier can be a small and simple trusted base. Proving the full semantic equivalence between source and target programs will be beyond the scope of this project. This project will implement a toolchain that uses type-preserving compilation to guarantee safety properties and statically verify the absence of certain unsafe behaviours and classes of runtime errors. A type-preserving compiler translates well-typed source code into verifiable target code, which for this project will be a typed assembly language. The typed assembly language will use features inspired by DTAL [4], where a register's type carries information about its value (e.g. the length of an array). Future work on this project could involve developing a formally verified compiler for the target assembly lan-